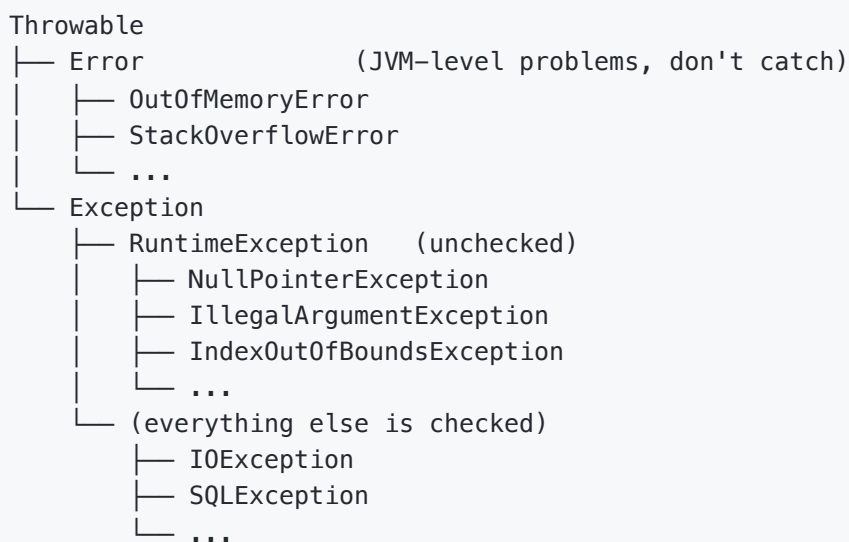


Java Exception Handling - Cheat Sheet

Exceptions are how Java signals that something went wrong at runtime. The language gives you a built-in hierarchy, a few keywords, and rules about who's responsible for handling what. Get the rules right and your error paths stay clean. Get them wrong and you'll spend hours debugging swallowed exceptions.

The Exception Hierarchy

Everything that can be thrown is a `Throwable`. Two main branches under it.



Quick rules:

- `Error`: JVM is in trouble. Don't catch these. Let the program die.
 - `RuntimeException` and subclasses: unchecked. Catching is optional.
 - Everything else under `Exception`: checked. The compiler forces you to handle or declare.
-

Checked vs Unchecked

Checked exceptions must be either caught or declared in the method signature. The compiler enforces this.

```
public void readFile() throws IOException {
    Files.readString(Path.of("data.txt"));
}
```

Examples: `IOException`, `SQLException`, `ClassNotFoundException`, `InterruptedException`.

Unchecked exceptions (anything under `RuntimeException`) don't need declaration. The compiler doesn't force you to handle them.

```
public int divide(int a, int b) {  
    return a / b; // throws ArithmeticException if b is 0  
}
```

Examples: `NullPointerException`, `IllegalArgumentException`, `IllegalStateException`, `ArrayIndexOutOfBoundsException`.

Rule of thumb: use checked exceptions for recoverable conditions the caller should handle (file missing, network down). Use unchecked for programming errors (null where it shouldn't be, bad arguments).

try / catch / finally

The basic three-block structure.

```
try {  
    int result = riskyOperation();  
} catch (IOException e) {  
    log.error("IO failed", e);  
} catch (SQLException e) {  
    log.error("DB failed", e);  
} finally {  
    cleanup();  
}
```

Rules:

- A `try` needs at least one `catch` or a `finally`.
 - Catch blocks are checked in order. Put more specific exceptions before general ones, or you'll get an unreachable-code compile error.
 - `finally` runs whether the `try` block completes normally, throws an exception, or returns early. Use it for resource cleanup.
-

try-with-resources

Java 7 added a cleaner pattern for anything that implements `AutoCloseable`. The resource closes automatically when the block exits.

```
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line = reader.readLine();
    System.out.println(line);
} catch (IOException e) {
    log.error("read failed", e);
}
// reader.close() called automatically
```

Multiple resources in one statement:

```
try (
    Connection conn = dataSource.getConnection();
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()
) {
    // use them
}
// closed in reverse order: rs, stmt, conn
```

Use this anywhere you'd previously write a `finally` block just to call `close()`. It's shorter, safer, and handles suppressed exceptions correctly.

throw vs throws

Two different keywords with similar spelling.

throw actually throws an exception:

```
public void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("age must be non-negative");
    }
    this.age = age;
}
```

throws declares that a method might throw a checked exception:

```
public String readConfig(String path) throws IOException {
    return Files.readString(Path.of(path));
}
```

You only need `throws` for checked exceptions. Unchecked exceptions can be declared too, but the compiler doesn't require it.

Multi-Catch

Java 7 added the ability to catch multiple exception types in one block when you'd handle them the same way.

```
try {
    doSomething();
} catch (IOException | SQLException e) {
    log.error("operation failed", e);
    throw new ServiceException(e);
}
```

The caught variable is implicitly `final` and typed as the common supertype. Don't reassign it.

Custom Exceptions

Build your own when the standard ones don't carry the right meaning.

```
public class InsufficientFundsException extends RuntimeException {
    private final double balance;
    private final double attempted;

    public InsufficientFundsException(double balance, double attempted) {
        super(String.format("balance %.2f, attempted %.2f", balance, attempted));
        this.balance = balance;
        this.attempted = attempted;
    }

    public double getBalance() {
        return balance;
    }

    public double getAttempted() {
        return attempted;
    }
}
```

Pick the parent carefully:

- Extend `RuntimeException` for unchecked (programming-error style).
- Extend `Exception` for checked (caller-must-handle style).
- Don't extend `Throwable` or `Error` directly.

Common pattern: include extra fields that explain what went wrong, beyond just a message.

Exception Chaining

When you catch one exception and throw another, attach the original as the cause. You'll thank yourself when debugging.

```
try {
    queryDatabase();
} catch (SQLException e) {
    throw new ServiceException("failed to load user", e);
}
```

The cause shows up in the stack trace under `Caused by:`. Without it, you lose the trail back to the original failure.

Every `Throwable` constructor can take a cause:

```
new RuntimeException("wrapper", originalException);
```

Retrieve it later if needed:

```
Throwable cause = caughtException.getCause();
```

Best Practices

- **Catch specific exceptions first.** Never start with `catch (Exception e)` when you only meant to catch one type.
 - **Don't catch what you can't handle.** If you don't know what to do with it, let it propagate.
 - **Never swallow an exception.** An empty catch block hides bugs forever. At minimum, log it with the stack trace.
 - **Don't use exceptions for control flow.** They're expensive (stack trace capture) and obscure the logic.
 - **Prefer specific exception types.** `NumberFormatException` tells you more than `RuntimeException`.
 - **Log the stack trace, not just the message.** `log.error("failed", e)` includes the trace. `log.error(e.getMessage())` doesn't.
 - **Close resources with try-with-resources.** Not with manual `finally` blocks.
-

Common Gotchas

- An exception thrown in `finally` overrides any exception thrown in `try` or `catch`. The original is lost unless you save it first.
 - `return` inside a `finally` block silently replaces a `return` or thrown exception from the `try` block. Don't do this.
 - Catching `Throwable` or `Exception` and swallowing it can hide `OutOfMemoryError` or `InterruptedException`. Both deserve special handling.
 - `InterruptedException`: if you catch it and can't propagate, re-set the interrupt flag with `Thread.currentThread().interrupt()`.
 - Autoboxing can throw `NullPointerException` silently: `Integer n = map.get(key); int x = n;` blows up if the key is missing.
 - Checked exceptions don't play well with lambdas. `Runnable.run()` can't throw checked exceptions, so you either wrap them as unchecked or use a custom functional interface.
 - `try-with-resources` calls `close()` even after an exception. If `close()` also throws, the original is preserved and the close exception is attached as "suppressed". Read them with `Throwable.getSuppressed()`.
 - A method that overrides another can't declare broader checked exceptions than the parent. It can declare fewer or narrower ones.
-

Quick Reference

Concept	Key fact
<code>Throwable</code>	Root of everything you can throw
<code>Error</code>	JVM-level, don't catch
<code>RuntimeException</code>	Unchecked, programming errors
<code>Exception</code> (other)	Checked, recoverable conditions
<code>throw</code>	Actually throws an exception
<code>throws</code>	Declares possible checked exceptions
<code>finally</code>	Always runs, even on exception or return
<code>try-with-resources</code>	Auto-close for any <code>AutoCloseable</code>
Multi-catch	One block for multiple exception types (Java 7+)
Exception chaining	Pass the original as <code>cause</code> to preserve the trail
Suppressed exceptions	Available via <code>Throwable.getSuppressed()</code>
Interrupted thread	Re-set with <code>Thread.currentThread().interrupt()</code>